bytebytego.com/join-10-of-1,000+

16
Design Google Drive

In this week's issue, we will walk through the steps of designing Google Drive, Microsoft OneDrive, and Apple iCloud. Cloud storage services are popular in the tech world, and they are used by billions of users.

Let's take a moment to understand how Google Drive works behind the scenes. Google Drive is the storage and synchronization service that allows users to store files in the cloud, access them from any device, and share them with others. It supports various file types, including documents, photos, videos, and more. Users can upload files to Google Drive, organize them into folders, and access them from anywhere with an internet connection.

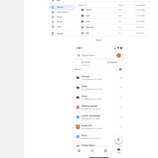

Figure 1

Step 1 - Understand the problem and establish design scope

Designing a large-scale distributed system is an involved task and questions alone take hours to answer. Let's start with a few questions to narrow the scope of the problem.

Candidate: Should we focus on file sharing and syncing?

Candidate: Yes, I'm OK with basic collaboration.

Interviewer: Sounds good.

Candidate: How many users are there?

Interviewer: Assume 10 million users.

Candidate: Is file size a concern for us?

Interviewer: Yes, we should support large files.

Candidate: How many files do we need to store?

Interviewer: Yes, that sounds like a lot.

Candidate: How many devices do each user have?

Interviewer: Two.

In this chapter, we focus on the following features:

- Upload files
- Download files
- Sync files across multiple devices. When a file is added to one device, it is automatically synced to other devices.
- See file revisions
- Share files with other users. They should be able to view/edit the file to collaborate with other users.
- Send a notification when a file is edited/deleted/shared with others.

Let's end with a back-of-the-envelope calculation.

- Assume 10 million users and 2 million active users.
- Assume 10 GB space allocated for each user.
- Assume 1 GB of data is uploaded to Google Drive per day.
- 1.2 peta-bytes (PB) of storage is needed: 10 * 10 * 1024 * 0.1 = 1.2 PB
- Assume upload and download QPS is 24 and 1 million, respectively. 1 * 1000 * 1000 / 24 = 41,667

Back of the envelope estimation

- Assume the application can hold 1 billion files (assume 1 million users and each user can store 1 GB data).
- Assume each file is 10 GB on average. This is a large file for a typical system.
- 1.2 peta-bytes (PB) of storage is needed: 10 * 10 * 1024 * 0.1 = 1.2 PB
- Assume upload and download QPS is 24 and 1 million, respectively. 1 * 1000 * 1000 / 24 = 41,667
- Assume read to write ratio is 1:1.

Step 2 - Propose high-level design and get buy-in

The following high-level diagram shows how uploaded files are stored in Google Drive. Users upload files to Google Drive and can access the uploaded files from any device. The application client is in charge of uploading files to the cloud storage. The application client notifies other clients about the file update.

Let's walk through each step in detail.

- Upload a file to a cloud storage.
- Download a file.
- Get file updates.
- Send file updates to other clients.

We agree that there will being a separate web server, a block/metadata server, and a notification server. Let's take a look at each component and see what it does. We will discuss why we need these components later in the chapter. Let's start with the upload flow. The upload flow is simple and we will discuss the two-step upload process later in the chapter. The upload flow is in charge of adding new files to the system and uploading new versions of the file to the cloud storage.

Figure 2 shows an example of how the client, blocking server, block files, cloud storage and cold storage work together to upload a file.

Figure 2

APIs

APIs are the bits that the general public sees. We upload files, download files, and get file revisions.

APIs for the Google Drive

Upload a file to Google Drive:

- Simple upload: Use this option if the file is small.
- Resumable upload: Use this option if you think the file is large and there's a high chance of network interruption.

Download a file:

- Download a file.

Get file revisions:

- Get file revisions.

Send file updates to other clients:

- Long poll.

Upload a file:

- Simple upload: Use this option if the file is small.
- Resumable upload: Use this option if you think the file is large and there's a high chance of network interruption.

Upload a file to Google Drive:

```
POST /upload/drive/v3/files?uploadType=resumable
Authorization: Bearer {access_token_for_google_drive}
Content-Type: application/octet-stream
Content-Length: {total_file_length_in_bytes}
X-Upload-Content-Type: {mime_type_of_file_content}
X-Upload-Content-Length: {total_file_length_in_bytes}
{file_metadata_in_JSON_format}
```

Download a file:

```
GET /drive/v3/files/{file_id}?alt=media
Authorization: Bearer {access_token_for_google_drive}
Range: bytes={start_byte_position}-{end_byte_position}
If-Range: {last_modified_timestamp_or_etag_of_file_content}
```

Get file revisions:

```
GET /drive/v3/files/{file_id}/revisions?fields=*
Authorization: Bearer {access_token_for_google_drive}
```

Send file updates to other clients:

```
GET /notifications/channels?projection=FULL&resourceId=my-drive&access_token=ya29.a0AfH6SMBx...&maxResults=100&alt=json
```

Move away from single server

As more files are uploaded, eventually we will run out of disk space. It is a bad idea to keep all files on one server.

Figure 3

Only 10 GB of storage space is used. This is a dangerous to store uploaded files anyone. The file system volume is close to a disaster. We decide to create a sharded storage system to store files across many servers.

Figure 4

We put all objects in an id, attribute-blobs key-value pair in a store. Sharding usually works pretty well. You have mapped the file id to a specific shard server. Sharding is a good way to distribute data across many servers. Sharding allows us to add more servers to the system when we need more storage. Sharding is a good way to distribute the load across many servers. Sharding allows us to add more servers to the system when we need more processing power. Sharding is a good way to distribute data across many servers. Sharding allows us to add more servers to the system when we need more processing power.

After a lot of thinking, we decided to use Amazon Simple Storage Service (S3) for our storage system. We could have created our own distributed storage system, but it would take a long time to get it right. We could have created our own distributed storage system, but it would take a long time to get it right. We could also have built our own distributed storage system, but it would take a long time to get it right. We could also have built our own distributed storage system, but it would take a long time to get it right.

Figure 5

After putting the S3, we can finally have a good night's sleep without worrying about data loss. We now have unlimited storage space. We can upload as many files as we want. We can upload as many files as we want. We can upload as many files as we want.

- Upload a file: Upload a file to Google Drive.
- Download a file: Download a file from Google Drive.
- Get file revisions: Get file revisions.
- Send file updates to other clients: Send file updates to other clients.

Amazon Simple Storage Service (S3) is a distributed object store that allows us to store and retrieve data. We can store and retrieve data using the S3 API. We can store and retrieve data using the S3 API. We can store and retrieve data using the S3 API. We can store and retrieve data using the S3 API.

Figure 6

Sync conflicts

For a large scale system like Google Drive, conflicts happen from time to time. When two users make the same file or folder at the same time, we need to resolve the conflict. We need to resolve the conflict. We need to resolve the conflict. We need to resolve the conflict.

Figure 7

In Figure 8, both users try to update the file at the same time. We need to resolve the conflict. We need to resolve the conflict. We need to resolve the conflict. We need to resolve the conflict. We need to resolve the conflict. We need to resolve the conflict.

```
/home/dropbox/photos/ - 2 updates
/home/dropbox/photos/cat.jpg - 1 update
```

Figure 8

Which folder was edited the latest documents in the system. It is challenging to keep the document synchronized between clients. Let's look at a few ways to solve the problem.

High level design

Figure 9 provides the overall high level design. Let's examine each component in the system.

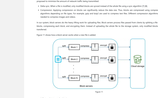

Figure 9

Client: A client application runs on the user's device and interacts with the cloud storage.

Block servers: Block servers upload files to cloud storage. Block servers offload files to cloud storage. Block servers are responsible for chunking the files, compressing the data, encrypting the data, and uploading the data to cloud storage. Block servers are responsible for chunking the files, compressing the data, encrypting the data, and uploading the data to cloud storage. Block servers are responsible for chunking the files, compressing the data, encrypting the data, and uploading the data to cloud storage.

Cloud storage: Cloud storage is a third-party cloud storage service. We use Amazon S3 for our application.

Cold storage: Cold storage is used for inactive data. Cold storage is used for inactive data. Cold storage is used for inactive data.

Load balancer: Load balancer evenly distributes requests among API servers.

API servers: Most operations are served by API servers. Most operations are served by API servers. Most operations are served by API servers.

Metadata database: Stores metadata of user files, folders, accounts, etc. Shards and data files are stored in the cloud and the metadata database keeps track of the metadata.

Metadata cache: Some of the metadata is cached for quick retrieval.

Notification server: It is a publish/subscribe system that allows clients to be notified that modified content is ready to sync (upload/download). For example, if a file is being edited on two clients, one client may need to be notified that the file is being edited on the other client, so that it can start downloading the updated version of the file.

Offline backing queue: It is a queue for offline sync and put the data through. The offline backing queue holds the list of changes until the client is ready to process them.

We have discussed the high level design. Let's discuss some of the critical components and design deep dives.

Step 3 - Design deep dive

In this section, we will take a closer look at the blocking, block servers, metadata database, upload flow, download flow, notification service, and high consistency requirements.

Block servers

An large file that will updates frequently, should be broken into smaller pieces and uploaded independently. Block servers are responsible for chunking the files, compressing the data, encrypting the data, and uploading the data to cloud storage.

Block servers work hard on compression, compressing small or medium sized files can often reduce the size dramatically. For example, a file in the text format may be reduced to a smaller size.

- Delta sync: Only the modified blocks are uploaded to cloud storage.
- Compression: Compressing compression of blocks can significantly reduce the data size. For example, text compressed using compression algorithms can often be reduced to less than half of the size. Image and video compression can also significantly reduce the size of the data.

In a system where clients for Windows, Mac, Linux, Android, and iOS, Google Drive supports the upload feature by splitting the file into fixed-size chunks. Google Drive supports the upload feature by splitting the file into fixed-size chunks. Google Drive supports the upload feature by splitting the file into fixed-size chunks.

Figure 10 shows how a file is split into smaller chunks and then uploaded to cloud storage.

Client
Block Server
Cloud Storage
Block Server
Block Server
Block Server